Session with FastAPI Dependency

Before we keep adding things, let's change a bit how we get the session for each request to simplify our life later.

Current Sessions

Up to now, we have been creating a session in each *path operation*, in a `with` block.

Python 3.10+

```python
# Code above omitted 👆

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate):
    with Session(engine) as session:
        db_hero = Hero.model_validate(hero)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)


class HeroCreate(HeroBase):
    pass


class HeroPublic(HeroBase):
    id: int


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate):
    with Session(engine) as session:
        db_hero = Hero.model_validate(hero)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(offset: int = 0, limit: int = Query(default=100, le=100)):
    with Session(engine) as session:
        heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
        return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
```

```python
        return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate):
    with Session(engine) as session:
        db_hero = session.get(Hero, hero_id)
        if not db_hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        hero_data = hero.model_dump(exclude_unset=True)
        db_hero.sqlmodel_update(hero_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero


@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        session.delete(hero)
        session.commit()
        return {"ok": True}
```

That's perfectly fine, but in many use cases we would want to use FastAPI Dependencies, for example to **verify** that the client is **logged in** and get the **current user** before executing any other code in the *path operation*.

These dependencies are also very useful during **testing**, because we can **easily replace them**, and then, for example, use a new database for our tests, or put some data before the tests, etc.

So, let's refactor these sessions to use **FastAPI Dependencies**.

Create a FastAPI Dependency

A **FastAPI** dependency is very simple, it's just a function that returns a value.

It could use `yield` instead of `return`, and in that case **FastAPI** will make sure it executes all the code **after** the `yield`, once it is done with the request.

Python 3.10+

```python
# Code above omitted 👆

def get_session():
    with Session(engine) as session:
        yield session

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)


class HeroCreate(HeroBase):
    pass


class HeroPublic(HeroBase):
    id: int


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
        yield session


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(*, session: Session = Depends(get_session), hero: HeroCreate):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    *,
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes
```

```python
@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    *, session: Session = Depends(get_session), hero_id: int, hero: HeroUpdate
):
    db_hero = session.get(Hero, hero_id)
    if not db_hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    db_hero.sqlmodel_update(hero_data)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.delete("/heroes/{hero_id}")
def delete_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

Use the Dependency

Now let's make FastAPI execute a dependency and get its value in the *path operation*.

We import `Depends()` from `fastapi`. Then we use it in the *path operation function* in a **parameter**, the same way we declared parameters to get JSON bodies, path parameters, etc.

Python 3.10+

```python
from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

# Code here omitted 👈

def get_session():
    with Session(engine) as session:
        yield session

# Code here omitted 👈

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(*, session: Session = Depends(get_session), hero: HeroCreate):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)


class HeroCreate(HeroBase):
    pass


class HeroPublic(HeroBase):
    id: int


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
        yield session


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(*, session: Session = Depends(get_session), hero: HeroCreate):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    *,
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes
```

```python
@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    *, session: Session = Depends(get_session), hero_id: int, hero: HeroUpdate
):
    db_hero = session.get(Hero, hero_id)
    if not db_hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    db_hero.sqlmodel_update(hero_data)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.delete("/heroes/{hero_id}")
def delete_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

Tip

Here's a tip about that `*,` thing in the parameters.

Here we are passing the parameter `session` that has a "default value" of `Depends(get_session)` before the parameter `hero`, that doesn't have any default value.

Python would normally complain about that, but we can use the initial "parameter" `*,` to mark all the rest of the parameters as "keyword only", which solves the problem.

You can read more about it in the FastAPI documentation
Path Parameters and Numeric Validations - Order the parameters as you need, tricks

The value of a dependency will **only be used for one request**, FastAPI will call it right before calling your code and will give you the value from that dependency.

If it had `yield`, then it will continue the rest of the execution once you are done sending the response. In the case of the **session**, it will finish the cleanup code from the `with` block, closing the session, etc.

Then FastAPI will call it again for the **next request**.

Because it is called **once per request**, we will still get a **single session per request** as we should, so we are still fine with that. ✅

And because dependencies can use `yield`, FastAPI will make sure to run the code **after** the `yield` once it is done, including all the **cleanup code** at the end of the `with` block. So we are also fine with that. ✅

The `with` Block

This means that in the main code of the *path operation function*, it will work equivalently to the previous version with the explicit `with` block.

Python 3.10+

```python
from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

# Code here omitted 👈

def get_session():
    with Session(engine) as session:
        yield session

# Code here omitted 👈

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(*, session: Session = Depends(get_session), hero: HeroCreate):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)


class HeroCreate(HeroBase):
    pass


class HeroPublic(HeroBase):
    id: int


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
        yield session


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(*, session: Session = Depends(get_session), hero: HeroCreate):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    *,
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes
```

```python
@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    *, session: Session = Depends(get_session), hero_id: int, hero: HeroUpdate
):
    db_hero = session.get(Hero, hero_id)
    if not db_hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    db_hero.sqlmodel_update(hero_data)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.delete("/heroes/{hero_id}")
def delete_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

In fact, you could think that all that block of code inside of the `create_hero()` function is still inside a `with` block for the **session**, because this is more or less what's happening behind the scenes.

But now, the `with` block is not explicitly in the function, but in the dependency above:

Python 3.10+

```python
from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

# Code here omitted 👈

def get_session():
    with Session(engine) as session:
        yield session

# Code here omitted 👈

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(*, session: Session = Depends(get_session), hero: HeroCreate):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)


class HeroCreate(HeroBase):
    pass


class HeroPublic(HeroBase):
    id: int


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
        yield session


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(*, session: Session = Depends(get_session), hero: HeroCreate):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    *,
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes
```

```python
@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    *, session: Session = Depends(get_session), hero_id: int, hero: HeroUpdate
):
    db_hero = session.get(Hero, hero_id)
    if not db_hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    db_hero.sqlmodel_update(hero_data)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.delete("/heroes/{hero_id}")
def delete_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

We will see how this is very useful when testing the code later. ✅

Update the Path Operations to Use the Dependency

Now we can update the rest of the *path operations* to use the new dependency.

We just declare the dependency in the parameters of the function, with:

```python
session: Session = Depends(get_session)
```

And then we remove the previous `with` block with the old **session**.

Python 3.10+

```python
from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

# Code here omitted 👈

def get_session():
    with Session(engine) as session:
        yield session

# Code here omitted 👈

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(*, session: Session = Depends(get_session), hero: HeroCreate):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero
```

```python
@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    *,
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    *, session: Session = Depends(get_session), hero_id: int, hero: HeroUpdate
):
    db_hero = session.get(Hero, hero_id)
    if not db_hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    db_hero.sqlmodel_update(hero_data)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.delete("/heroes/{hero_id}")
def delete_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)


class HeroCreate(HeroBase):
    pass


class HeroPublic(HeroBase):
    id: int


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
        yield session


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(*, session: Session = Depends(get_session), hero: HeroCreate):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    *,
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes
```

```python
@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    *, session: Session = Depends(get_session), hero_id: int, hero: HeroUpdate
):
    db_hero = session.get(Hero, hero_id)
    if not db_hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    db_hero.sqlmodel_update(hero_data)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.delete("/heroes/{hero_id}")
def delete_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}
```

Recap

You just learned how to use **FastAPI dependencies** to handle the database session. This will come in handy later when testing the code.

And you will see how much these dependencies can help the more you work with FastAPI, to handle **permissions**, **authentication**, resources like database **sessions**, etc. 🚀

If you want to learn more about dependencies, checkout the FastAPI docs about Dependencies.